



SCHOOL OF COMPUTATION,
INFORMATION AND TECHNOLOGY -
INFORMATICS

TECHNICAL UNIVERSITY OF MUNICH

Bachelor's Thesis in Informatics: Games Engineering

**Communication Interfaces for Distributed
Augmented Reality Applications in
Industrial Contexts**

Maximilian Alexander Amougou Mbida



SCHOOL OF COMPUTATION,
INFORMATION AND TECHNOLOGY -
INFORMATICS

TECHNICAL UNIVERSITY OF MUNICH

Bachelor's Thesis in Informatics: Games Engineering

**Communication Interfaces for Distributed
Augmented Reality Applications in
Industrial Contexts**

**Kommunikationsschnittstellen für verteilte
Augmented Reality Anwendungen in
industriellen Kontexten**

Author:	Maximilian Alexander Amougou Mbida
Supervisor:	Prof. Gudrun Klinker, Ph.D.
Advisor:	Linda Rudolph, M.Sc.
Submission Date:	15. April 2023

I confirm that this bachelor's thesis in informatics: games engineering is my own work and I have documented all sources and material used.

Munich, 15. April 2023

Maximilian Alexander Amougou Mbida

Acknowledgments

I would like to express my sincere gratitude to my supervisor, Linda Rudolph, for her unwavering support and guidance throughout my research. Her expertise and insights were invaluable in shaping the direction of my thesis.

In addition, I would like to thank everyone who participated in my user study.

Finally, I would like to thank my family and friends, who provided emotional support and encouragement throughout my academic journey.

Abstract

This thesis explores the use of augmented reality (AR) technology in industrial contexts and proposes a solution to the problem of sharing 3D volumes of interest across distributed computing and communication networks. The main goal is to develop a system that allows users to send and receive 3D volumes of interest in a lightweight format, specifically the glTF format, using AR technology. The thesis provides an overview of the technical background, including Vuforia extended tracking, voxel carving, Marching Cubes algorithm, and glTF. The related work section discusses similar uses of AR and glTF in industrial contexts. The methodology section presents the conceptual framework and implementation details, including voxel carving, Marching Cubes algorithm, and glTF export and import. A non representative user study evaluates the system's effectiveness and usability regarding the newly implemented communication interface. The test scores showed promising results indicating no severe usability deficiencies in the system and the usefulness of the implementation to industry professionals. The contribution of this thesis enables professionals in the industry to work collaboratively and remotely on projects and to connect on-site collected information to an information model, leading to improved productivity and reduced costs. Additionally, the app's ability to create 3D volumes of interest has numerous applications in various industries, including healthcare, architecture, and engineering. This has the potential to bring substantial value to the industry and generate further research interest in this field.

Contents

Acknowledgments	iii
Abstract	iv
1. Introduction	1
1.1. Use case and problem statement	1
1.2. Goals of the thesis	2
2. Fundamentals and Concept	3
2.1. Augmented reality technology	3
2.2. Distributed computing and communication	3
2.3. Technical background	4
2.3.1. Vuforia extended tracking	4
2.3.2. Voxelcarving	4
2.3.3. Marching Cubes algorithm	5
2.3.4. glTF	6
2.4. Concept summary	6
3. Related Work	9
3.1. AR in industrial context	9
3.2. glTF for 3D object sending	9
4. Methodology	11
4.1. Conceptual framework	11
4.2. Implementation	11
4.2.1. Voxelcarving	11
4.2.2. Marching Cubes algorithm	12
4.2.3. glTF export and import	14
4.2.4. Test server	16
4.3. Additional features and improvements	17
4.3.1. Settings tab improvement	17
4.3.2. User interface elements	18

5. Evaluation	19
5.1. Setup	19
5.2. Results and evaluation	22
5.2.1. Demographic survey	22
5.2.2. Observations	22
5.2.3. System usability scale	23
5.2.4. NASA task load index	23
5.2.5. Application survey	24
5.2.6. Interview	27
5.2.7. Summary	28
6. Future Work	29
7. Conclusions	30
A. Added user interface elements	31
List of Figures	33
List of Tables	34
Bibliography	35

1. Introduction

1.1. Use case and problem statement

Large industrial companies are often located in multiple and distant locations, making it difficult for experts to inspect and assess facilities remotely. One solution is to create CAD models of the facilities, allowing experts to remotely inspect and guide maintenance workers on site. However, these models quickly become outdated due to regular modifications and repairs [Rud+22]. To solve this problem, a new approach has been proposed: creating a 1:1 digital reconstruction of individual parts and machinery within the facility and integrating them into the existing CAD model [RK21]. In essence, the aim has been to generate a digital replica of the facility, also known as a digital twin.

To simplify this, a 3D annotation method has been created, which involves taking pictures of an object from different angles and tracing its edges to then create a low-detail digital 3D reconstruction with the help of voxel carving. This 3D object should then be sent to a server, so that it can be integrated in the CAD model of the facility and the expert can access the result remotely without having to travel to the facility. The server is necessary because the CAD model consumes so much memory that the users device (a tablet computer) cannot handle it.

An app in which the user could use image annotation and voxel carving to create the Volume of Interest (VoI) already exists [Sti22] [Klu23]. Currently the VoI is only usable locally on the device on which the app is running meaning it is augmented in the scene but nothing else can be done with it.

The use case states a scenario where a worker in a industrial facility annotates the modified part of the machine for an expert so he can work with it remotely so he does not have to come to the facility himself. For that we have to be able to send the VoI from the workers device, to the experts device. One challenge hereby is that we do not only want to send the generated 3D object but also its exact location in the machine, so the expert can locate it on the CAD model and continue the maintenance of the digital twin.

In conclusion, we want to establish a communication interface between the app and a server, for exchanging this type of information, so that the worker can upload the VoI and the expert can download it later. Furthermore at the moment the created VoI is

created through voxelcarving and each valid voxel is represented by a semi transparent yellow cube template, so the VoI is represented by a cloud of cubes which sometimes leads to opaque volumes when there are too many cubes behind each other. This makes it impossible to recognize the original object. Plus, this representation of the volume consumes more memory than necessary, since many unnecessary corner points have to be stored. Therefore, we want to find a better representation of the volume.

1.2. Goals of the thesis

The goals of this thesis project are threefold. Firstly, implementing the interface to send the VoI and its pose to a server using the .glTF data format, allowing for efficient data transfer and processing. Secondly, to incorporate the Marching Cubes algorithm into the application, providing a cleaner and less memory-intensive representation of the VoI. Finally, the project aims to investigate whether there is a significant difference in test scores or other metrics when switching from an iPad to an Android device and adding the previously mentioned features to the application. These goals will be achieved through the development and implementation of appropriate software tools and testing procedures, as well as the analysis and interpretation of the resulting data.

2. Fundamentals and Concept

2.1. Augmented reality technology

Augmented reality (AR) technology has rapidly developed over the last few years, with a growing number of applications in various industries, such as healthcare, education, and entertainment [Zhu+14] [BD12] [Cho+18]. The technology has the potential to provide users with an immersive and interactive experience that goes beyond what is possible with traditional screens or displays. According to Azuma (1997) AR is any system with the following characteristics:

1. Combines real and virtual
2. Is interactive in real time
3. Is registered in three dimensions

[Azu97].

Spatial relationship graphs play a critical role in this process, as they provide a way to model the relationship between the user, the real-world environment, and virtual objects [Ech+08]. These graphs enable AR systems to understand the spatial context of the environment and accurately place and track virtual objects in real time (see 2.1). This spatial awareness is achieved through various sensors, such as cameras, depth sensors and GPS, which capture data about the environment and the user's position. By combining this data with spatial relationship graphs, AR systems can create a compelling and realistic experience for users.

2.2. Distributed computing and communication

Distributed computing and communication refers to the use of multiple networked computers to work together on a common task. In distributed computing, a task is divided into smaller parts, which are then processed by different computers simultaneously. This allows for faster processing and more efficient use of computing resources. Communication in a distributed computing system is facilitated through various protocols and technologies that enable different computers to exchange data and coordinate

their actions (see 2.2, 2.3). Examples of distributed computing systems include cloud computing platforms, peer-to-peer networks, and grid computing environments. Distributed computing and communication have become increasingly important, as they allow for the processing and storage of large amounts of data and the execution of complex tasks that would be impractical or impossible for a single computer.

2.3. Technical background

2.3.1. Vuforia extended tracking

Vuforia extended tracking is a feature of the Vuforia augmented reality platform that enables the application to keep track of the target's pose information even if the target is no longer visible for the camera. Essentially, it allows the app to display the augmented objects in the right place and orientation. Therefore, the user can move freely in space and view the augmented 3D object from different positions and does not have to make sure that the marker is visible to the camera, resulting in a more natural and immersive user experience [PTC21]. Vuforia extended tracking is particularly useful for applications such as product visualization, where users might want to view a product from different angles, or educational applications, where students might want to explore a virtual model in greater detail. It is also valuable in industrial applications, where it can be used to overlay information on real-world objects, as in our use case. In this scenario, the user creates a VoI to verify the accuracy of their annotations, and to ensure that they can be successfully converted into the VoI that they intend to send to the server.

Overall, Vuforia extended tracking is an important feature of the Vuforia platform that enables more immersive and engaging augmented reality experiences.

2.3.2. Voxelcarving

Voxelcarving is a technique used in 3D computer graphics and computer vision that involves creating a 3D model of an object or scene from a set of 2D images. The basic idea behind voxelcarving is to create a volumetric representation of the object or scene by dividing the space into small 3D elements known as voxels. The process of voxelcarving typically involves two steps: reconstruction and refinement.

During the reconstruction step, the 2D images are used to create an initial 3D model by carving out the space occupied by the object in each image. As first step a 3D bounding box is calculated. This bounding box then gets divided in $X*Y*Z$ (dimension sizes of the cube) equally large 3D voxels. Each voxel v is then tested if it is inside or

outside the object we want to reconstruct. This creates a rough approximation of the object's shape in 3D space [Che+00].

During the refinement step, the model is iterative improved by subdividing the valid voxels into 8 smaller voxels and testing the newly created ones again. Resulting in a more accurate representation of the real world object.

Voxelcarving has applications in a wide range of fields, including computer graphics, computer vision, and robotics. It is commonly used in augmented reality and virtual reality applications, where it can be used to create realistic 3D models of real-world objects and scenes. One advantage of voxelcarving is that it can be used to create 3D models from a relatively small number of 2D images, which makes it an attractive option for our application where obtaining a large number of images is difficult or impractical [Che+00] [Sti22]. The usefulness of voxelcarving in our use case has been explored by [Klu23] in a casestudy. According to Klusmann (2023) his "[...] study [...] provide[s] strong evidence supporting the use of voxelcarving as a volume of interest annotation method." [Klu23] (p. 47)

In summary, voxelcarving is a powerful technique for creating 3D models from 2D images. Its ability to create accurate and realistic 3D models from a small number of images makes it a useful tool for our usecase.

2.3.3. Marching Cubes algorithm

The Marching Cubes algorithm is a popular method for generating 3D surface meshes from volumetric data. It was first introduced by William E. Lorensen and Harvey E. Cline in 1987, and has since then become a standard technique in the field of computer graphics and scientific visualization. The algorithm works by partitioning a 3D scalar field into a regular grid of cubic cells. For each cell, the algorithm determines which of 256 possible configurations of the scalar values at the eight corners of the cube are present. Based on this configuration, the algorithm generates a polygonal approximation of the surface that passes through the cell. To generate the polygons, the algorithm uses a lookup table that contains information about the topology of the surface for each possible configuration of the scalar values. The table maps each configuration to a set of triangles that form the surface. By using this lookup table, the algorithm can generate a smooth, continuous surface that approximates the actual surface of the object being represented by the scalar field. One of the strengths of the Marching Cubes algorithm is its ability to handle complex, irregularly shaped surfaces. Because the algorithm generates a polygonal mesh, it can represent surfaces with holes, tunnels, and other features that would be difficult to represent with other methods [LC87].

Overall, the Marching Cubes algorithm is a strong tool for generating 3D surface

meshes from volumetric data. Its versatility and ease of implementation have made it a popular choice for a wide range of applications, including medical imaging, scientific visualization, computer graphics and also our application.

2.3.4. glTF

glTF (short for Graphics Language Transmission Format) is a file format designed for efficient transmission and loading of 3D scenes and models. It was developed by the Khronos Group, a consortium of companies and organizations that develops open standards for the graphics industry. glTF is designed to be a compact, efficient, and interoperable format that can be used with a wide range of 3D graphics applications and tools. It is based on a subset of the JSON (JavaScript Object Notation) data format and is designed to be easy to parse and process by web browsers and other software. An important feature of glTF is its support for binary data, which allows for more efficient transmission and loading of 3D models. glTF also supports embedded textures, which means that textures can be included directly in the glTF file, rather than having to be loaded separately [Mar22]. Overall, glTF is a versatile and efficient format for 3D graphics that is well-suited for use on the web and in other applications where size and efficiency are important factors.

2.4. Concept summary

In the app we use augmented reality technology to display the Volume of Interest which we have as a result of the previous voxelcarving directly in the real scene, so we can see the annotation in real life. For that we have a marker near the scanned object which represents the origin of the world coordinate system. With this marker we can calculate the devices position in the world and, respectively, the position of the VoI. After calculating the position in worldspace, we need to transform this position into screen coordinates so we can display the VoI on our background video. The following spatial relationship graph models this before mentioned setup (see 2.1).

Furthermore we want to establish a communication interface for sending the VoI from the tablet to an external server. We need to do this because the device does not have the computational power to do the further calculations by itself. That calculation includes integrating the VoI in a CAD model. Since CAD models of large industrial machines require a lot of memory [KLM20], processing is only possible with the help of a server.

Our reconstruction process employs voxelcarving to convert annotated images into a 3D object, or VoI. The resulting voxels are then processed by the Marching Cubes algorithm to create a polygonal mesh that can be augmented in the scene for the user

to verify if the annotation quality is sufficient. Vuforia extended tracking provides us with the necessary pose information of the device and the VoI relative to the marker, enabling us to add the augmentation to the scene and gather spatial data for the voxelcarving process. Finally, to ensure efficient delivery of the VoI, we have chosen glTF as our file format due to its numerous benefits like the support of binary data and the interoperability across a wide range of 3D graphics applications and tools [Lin].

3. Related Work

As stated earlier, this thesis is based on the groundwork of Stieglbauer [Sti22] and Klusmann [Klu23]. The project diligently explored effective solutions for our particular use case, ultimately leading to the adoption of voxel carving as the 3D reconstruction method. This chapter will outline similar applications of AR in industrial context and usages of glTF for 3D object sending.

3.1. AR in industrial context

In 2008, Schwerdtfeger and Klinker proposed the application of augmented reality to assist workers in the order picking process [SK08]. Their approach involved displaying visual cues on a head-mounted display (HMD). They conducted three test series and an exhibition trial with participants. The results indicated that AR annotation had the potential to reduce errors and picking times in the order picking process.

This paper demonstrates the use of AR technology to guide workers and streamline workflow. This application is similar to the objective of this thesis, which aims to employ AR annotation to guide workers who are not physically present in the factory. However, a key difference lies in the annotation process. In Schwerdtfeger and Klinker’s study, the annotation was automated, whereas in this thesis, the worker manually creates and adds the augmentation. Another notable difference is the device used, as Schwerdtfeger and Klinker utilized an HMD, while this thesis employs a handheld tablet computer with a built-in camera.

3.2. glTF for 3D object sending

In their 2016 research paper [SBN16], Schilling et al. proposed an approach for streaming large and highly detailed 3D city models efficiently, based on open standards and open source developments. The proposed approach involved several steps, including the subdivision of the 3D city model into tiles in a R-tree structure, selection of the Level of Detail (LoD) and textures, conversion of the CityGML geometries from polygons to triangles, conversion of coordinates from a Coordinate Reference System (CRS) to a local cartesian coordinate system, generation of a texture atlas, and mapping of

attributes and semantics. The researchers noted that the glTF format is limited to visual elements that can be handled by OpenGL and that B3DM is an extended version of glTF that can store additional data needed for displaying city models. Comparing glTF/B3DM/3D Tiles to other 3D formats (MeshLab/COLLADA, Google Earth/KMZ, MeshLab/X3D, X3DOM/X3D), the researchers found that the format can keep up with the latter while using open standards and open source development. However, for this use case of storing a mesh with its position, the information stored by glTF is sufficient, and using B3DM is unnecessary in this paper. It is also worth noting that the mesh creation process differs from that presented in this paper, as our presented use case requires creating a mesh from scratch and in [SBN16] it is converted from an existing mesh.

Miao et al. [MSZ17] explored the possibility on how to create an interactive experience of 3D visualisation of city models in web because normally these "3D city scenes have a great amount of data, including tens of thousands of buildings", meaning that users have to wait a long time to download these scenes. Furthermore "the whole scene cannot be called into the system memory [...] and the GPU cannot render in real-time". "In the framework, the web server side mainly provides data services and stores the 3D scene data in glTF format" (Miao et al. 2017). They came to the conclusion that "glTF is very suitable for Web exchange and rendering" and that in combination with a prefetching strategy and occlusion culling technology the bandwidth usage can be reduced and the visualization efficiency can be improved. This is also relevant for the use case presented in this paper as we want a efficient method for exchanging the VoIs.

4. Methodology

The "3D Reconstruction" application is a tool that enables users to annotate a volume of interest in real-world space through voxel carving and generate its corresponding 3D reconstruction in digital space. Developed with C# and Unity, the application is compatible with all major platforms, including iOS/iPadOS, Android, and Windows. In this chapter implementation of the before mentioned communication interfaces (see 2.2, 2.3), as well as the newly added features and improvements to the app will be explained.

4.1. Conceptual framework

As outlined in the introduction [1], the main objective was to enhance the application by enabling users to send annotations to a server, allowing remote workers to integrate changes made to the initial scan into the CAD model. In order to create a send- and processable VoI several steps needed to be done by the application:

1. Allow user to take photos of the object of interest and annotate them
2. Voxelcarving to receive the valid voxels from the annotation
3. Create a mesh from the carved voxels and augment it into the scene
4. Convert the mesh and it's pose respective to the marker into a .gltf file
5. Send the .gltf file to the server or download a previously created .gltf file from the server and validate the sent file

To achieve this, several new plugins and features were added to the application. Furthermore a python server was implemented to simulate the real world use case.

4.2. Implementation

4.2.1. Voxelcarving

During the further development of the app the voxel carving process was improved. The rough course of the algorithm is as follows. The algorithm gets triggered by a

reconstruct button, which the user can click if enough images (at least three) were made and all images were annotated.

First the *Smart Cube Adjustment* (SCA) implemented by [Klu23] calculates the coordinates of an approximated 3D bounding box which contains the Object of interest. This bounding box can also be augmented into the scene by enabling the corresponding setting but this is rather for debug purposes than for the common usage of the application.

Then with the volume of the box and the *Initial Voxel Count* setting, the start voxelsize is calculated and the box gets filled with voxels. The voxels get stored in a 3D boolean array which represents the box filled with voxels.

The next step is the actual carving process. The users can select the number of times the voxels get subdivided. In each iteration every voxelcenter gets checked if it is inside of every marking the user created. These are then consider valid. With these valid voxels the Marching Cubes algorithm gets initialized.

Significant improvements were made to the voxel carving process by checking only the voxel centers instead of all eight corners. This change led to a considerable increase in the speed of the intersection process, resulting in a more efficient workflow. Previously, voxels were considered valid only if they were border voxels, meaning at least one corner was outside and one inside. However, now every inside and border voxel is considered valid since the inner, non-visible voxels are removed later in the Marching Cubes algorithm. This method maintains accuracy while improving processing speed.

It is acknowledged that speed and accuracy can be a trade-off in some applications. To evaluate the changes, a comparison was made between the old (yellow, approximately 10 seconds) and new (blue, approximately 2 seconds) annotation methods, which were then merged (see 4.1). The comparison shows that processing speed increased significantly without sacrificing much accuracy.

4.2.2. Marching Cubes algorithm

The first two steps were already possible before the beginning of this work. The valid voxels where also already augmented into the scene. To visualize the valid voxels a yellow semitransparent cube prefab was used to represent each voxel. This lead to the problem that if there were too many voxels behind each other, the original object couldn't be seen very well (see 4.2). Another disadvantage of this approach was that the file size of the VoI would get unnecessary large, because for each valid Voxel 8 vertices had to be stored, while most of them weren't visible because they were on the inside. This would drastically increase the up- and download speed of the application. This approach was suitable for a first prototype but is not sufficient for the use in an



Figure 4.1.: In-app screenshots with old, new and merged VoIs

industrial context. To improve the appearance of the VoI, the Marching Cubes algorithm was added to the application. The main part of the algorithm was implemented in an Unity plugin by Justin Hawkins [Haw22] and is based on the work of William E. Lorensen and Harvey E. Cline [LC87]. In order to incorporate this plugin into the application, it was necessary to modify the voxel carving algorithm. The creation of a mesh from the voxels necessitated obtaining the voxel neighborhoods, which required a change in the storage format of the voxels. Originally there was just a simple list consisting of Vector3 elements, where each element represented the position of a valid voxel. Vector3 is a data structure used in Unity for storing positions and directions in 3D. It consists of three float values where each one represents one axis (x, y, z). In order to keep track of the voxel neighborhoods, a switch was made from using a Vector3 list to a 3D boolean array [x, y, z]. The decision to use a boolean array instead of a Vector3 array was based on memory efficiency, as boolean arrays require less space. A boolean usually requires 1 byte, while each component of a Vector3 is represented as a single precision floating point number, which requires 4 bytes of memory. So, 4 bytes * 3 components = 12 bytes [Uni]. Now a valid voxel is represented by the value "true".

To be able to calculate the position of each voxel, the start x, y and z coordinates of the *smart cube* are stored in the settings. The markerPosition, localScale and rotation-Quaternion are properties of the marker and are calculated by the vuforia extended tracking feature. The voxel size is calculated depending on the *Initial voxel count* setting. The following equation demonstrates the position calculation of the voxel with index



Figure 4.2.: VoI with cube prefabs

[5, 7, 3]:

$$\begin{pmatrix} x \\ y \\ z \end{pmatrix} = \begin{pmatrix} startX + 5 * voxelSize \\ startY + 7 * voxelSize \\ startZ + 3 * voxelSize \end{pmatrix} + \vec{markerPos} \Bigg) * rotationQuaternion * local\vec{Scale} \quad (4.1)$$

With the neighborhoods stored the plugin could almost be used. The last step was to convert the 3D boolean array to a voxelarray. This so called voxelarray is part of the Plugin and is represented by a 3D float array, where 0.5f represents inside/valid voxels and -0.5f represents outside/invalid voxels. The surface value was set to be -0.49f, so that the mesh size was maximised and would represent the annotations as good as possible. After the Marching Cubes algorithm is done, a new gameobject (a container for visual and behavioral components in the Unity game world) was created with the resulting mesh as component and set as child of the marker to place it in the right place in the world space. Now the VoI is augmented into the scene. The visual improvement this addition made can be seen in an in-app screenshot (4.3).

4.2.3. glTF export and import

The fourth step is necessary because even though it is already possible to import and export game objects in Unity, these files are not useful because they can only be used



Figure 4.3.: VoI with Marching Cubes algorithm

in an Unity workaround. This application and the created VoIs should be generally accessible and be compatible to the mainly used systems. Therefore the .gltf file format is the best solution. To add the import and export to .gltf files two additional plugins were used.

For exporting the mesh, the plugin UnityGLTF by Robert Long [Lon23] was used. This plugin is a fork from the UnityGLTF plugin developed by the Khronos Group. To use this plugin, no preprocessing of the data is necessary. The in step 3 created gamobject is exported to a by the application specified path. Meaning the VoI will be stored on the users device and replace the old one if there is already a .gltf file at this path. The export creates two files. One .gltf and one .bin file. The .gltf file is in JSON format and contains the description of the gameobject (name, colorinformations, etc.) and a description on how to parse the .bin file correctly as well. The .bin file contains information like coordinates, normals, indices, etc. saved as bytes.

For the import, the plugin glTFast by Andreas Atteneder [Att22] was used. The import functionality is needed, because in the application the user should not only be able to upload the created VoIs but also be able to download VoIs which were created by other users. Furthermore for experimental purposes the import was also used to verify the sending process. To import a .gltf file the plugin needs an already in the scene existing gameobject and the filepath or URL where the file is located. In this case the marker was used as gameobject. The only thing that needed to be done was to add



Figure 4.4.: Scene with server answer

a *GLTFast.glTFAsset* component to the marker and specify the location of the .gltf file. The rest was done by the plugin.

4.2.4. Test server

To simulate our real world use case a local-host test server was implemented with python. The server has two types of calls. POST calls to upload the .gltf and .bin file [2.2] and GET calls to download the .gltf and .bin file [2.3]. The GET calls send the latest uploaded files as answer to the client. The POST calls store the uploaded files by overwriting the old files. To give the client feedback if the export and upload was successful, a small transformations are done to the VoI and then sent back as answer to the client. The transformations consisted of mirroring the VoI across the yz-plane and changing it's color from yellow to blue [4.4]. The mirroring was done by parsing the .bin file how it was specified in the .gltf file and inverting each x-coordinate byte. Changing the color can be directly done inside the .gltf file by explicitly setting the [r, g, b, a] color. The values need to be in the range [0; 1] and [0.0, 0.7, 0.5, 0.35] were chosen, which resulted in a light-blue transparent VoI. This answer is also imported into the scene so that the user can see two VoIs and can verify that everything went well.

These transformations also demonstrate the re-usability of the exported VoI gltf.

4.3. Additional features and improvements

The main goal of this thesis is a proof of concept, to show that it is possible to integrate the application in an industrial context by implementing a new communication interface. Along the way several existing issues were resolved and improvements made.

4.3.1. Settings tab improvement

For the regular use of the app, you only need to adjust 2 settings. Most of the time, however, it works without any adjustments. The adjustments you can make are *initial voxel count* and *subdivision iteration count*.

Initial voxel count is the amount of voxels the bounding box from the SCA gets filled with. Increasing this number results in smaller startvoxels and an increased accuracy in the reconstruction process, but also in longer calculation times. *Subdivision iteration count* is the number of times each voxel is getting subdivided. This means in one iteration each voxel is split into 8 smaller voxels. So in each step the resolution of the annotation is increased resulting in a better reconstructed object. These settings are necessary because different users have different requirements for the reconstruction. If it is a low detail object for example a straight pipe the numbers can be chosen very low also resulting in a very quick calculation. If the object of interest has many details or even is convex, the values should be increased which also increases the computation time. In Conclusion, there is a trade off between reconstruction accuracy and reconstruction speed where the user must weigh up what is more important.

The settings *x,y,z start and end value for the cube* were made obsolete by the implementation of the SCA by Jack Klusmann [Klu23].

Another setting that was obsoleted during the work of this thesis was the *initial voxel size*. The voxelsize is now calculated automatically and depends on the *initial voxel count* setting. This was necessary to stabilize the system. The problem at hand was that users lacked knowledge on how to properly set the size. Often resulting in too small startvoxels which resulted in a huge amount of voxels that led to extreme slowdown of the system and even system crashes.

All obsoleted and not for the general use settings are now accessible with an *expert settings* button inside the settings tab, so that the general user can focus on the relevant aspects and does not get confused. Only developers and users with high knowledge on how the app works internally should adjust these.

4.3.2. User interface elements

Several user interface elements were added to the application for a more intuitive and more efficient usage. They are described in the appendix A.

5. Evaluation

Within the scope of this thesis, the usability tests of [Klu23] were carried out as a preliminary study which should not be seen as representative. The background was the question of whether the changeover from iPadOS to Android and the new (UI-) features would have a negative, positive or no impact on the previous test results. Overall, we wanted to uncover serious UI deficiencies. The study of Jakob Nielsen and Thomas K. Landauer [NL93] showed "A mathematical model of the finding of usability problems". Indicating that even a small number of study participants is sufficient to detect serious usability errors.

For this small test we also switched from landscape to portrait mode, as Klusmann stated it could be an improvement to the application and instead of an active touch-pen (Apple Pencil) a simple touch-pen was used to annotate the images because in an industrial use the tablets which are used are in a special cover with which the active pens would no longer work.

Model name:	Samsung Galaxy Tab S7+
Model number:	SM-T970 (Wi-Fi only)
Android version:	13
Launch year:	2020
Display size:	12.4 inches
Aspect ratio:	16:10
Weight:	575 g (without cover)

Table 5.1.: Test device information

5.1. Setup

The structure and procedure of the test was as follows:

1. User signs the consent form provided by the department of Augmented Reality
2. User fills out the demographic survey to gather information about age, gender, academic background and also previous experiences using a tablet device

and a touch-pen as well as previous experiences involving augmented reality applications [Klu23]

3. Brief introduction to application and the use case by test supervisor
4. Application demonstration by test supervisor on an example object
5. The test participant gets the possibility to ask questions
6. The participant should rate how well prepared they felt to use the application after the introduction and demonstration on a scale from 1 (least prepared) to 10 (most prepared).
7. Usability testing begins
 - a) User is asked to create reconstruction of three objects (see 5.1) and the test supervisor measures time, notes objects size, errors and other observations (e.g. amount of help needed by supervisor)
 - b) User scans the marker to get the tablets pose with respect to the marker
 - c) User takes picture of the object of interest on tablet
 - d) User creates carving using touch-pen right after taking picture
 - e) User repeats this process until 3 valid pictures and carvings have been created
 - f) User initiates intersection - intersection might fail if images or carvings are not sufficiently good
 - g) User judges the quality of the result and is asked whether they are satisfied with the result and if not given chance to retake images and redo carvings if necessary
 - h) The object with which the user had the most problems or for which he took the longest time should be repeated a second time in order to see if there was a learning process
8. User fills out the System Usability Scale (SUS) survey [Bro+96]
9. User fills out NASA Task Load Index (TLX) survey [Har86]
10. User fills out application specific survey which focuses on features and UI elements of this application [Klu23]
11. Test Supervisor holds interview with user where the participant could explain difficulties while using the system and as well give suggestions for the further improvement of the application (questions from [Klu23])

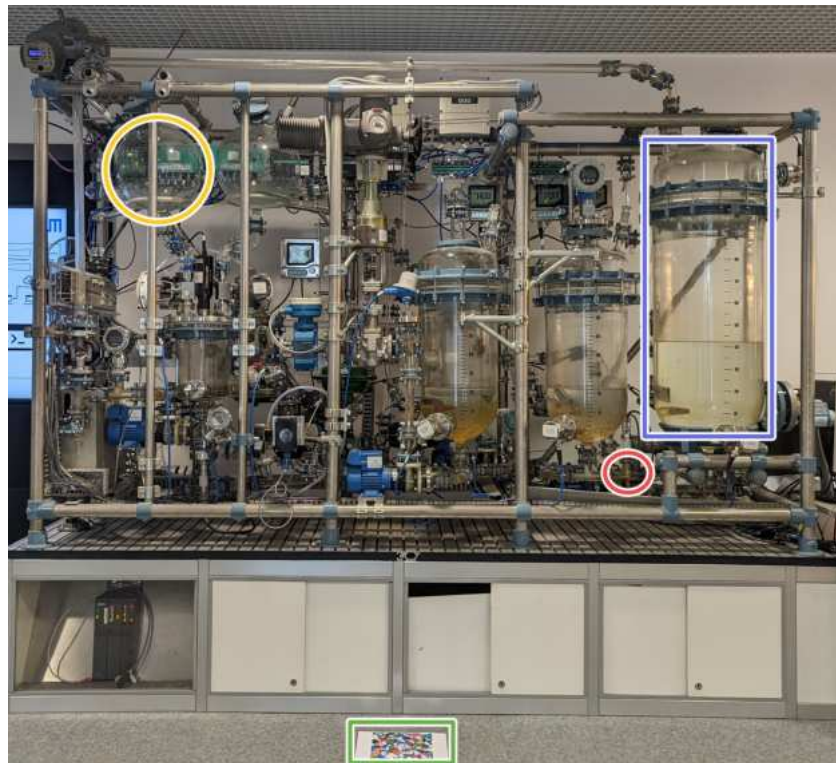


Figure 5.1.: Test machine with the marker (green) and three different sized objects of interest: small (red), medium (yellow), large (purple)

5.2. Results and evaluation

The user tests were performed on the same machine as Klusmanns usability tests and the objects of interest were also the same. Students from the TUM School of Computation, Information and Technology the Ludwig-Maximilian-University were invited to participate in the tests. In total there were 3 participants.

5.2.1. Demographic survey

All participants are between the ages 22 and 25 with a male:female ratio of 2:1. All participants are pursuing a bachelors degree. One in mathematics and one in informatics background and one with no technical background. One participant owns a tablet device with a stylus which is used more than three times a week. Two participants had little to some experience with augmented reality technology which they use from less than once a month up to 3 times a month and the other one not at all.

5.2.2. Observations

The confidence level of the participants after the introduction and demonstration were 7 and 9 out of 10 so the participants were feeling well prepared. The average time to annotate the three objects (small, medium, large) were 307, 93, 97 seconds which is comparably just as fast as in the previous tests by Klusmann. One participant forgot to scan the marker before the first task. One participant forgot to hit the reload button to start annotating the next object. While having problems to scan the small object one participant falsely decreased the initial voxel count instead of increasing.

The long average time for the small object resulted from the fact that the object was more occluded than the other two and therefore it was more difficult to take pictures from different angles. This caused the Smart Cube Adjustment to fail multiple times. Another reason for the longer duration was that the settings had to be adjusted to get a result. The default values *Initial Voxel Count* (100) and *Subdivision Iteration Count* (3) were not sufficient and needed to be increased. The small object was the only object where the test supervisor had to assist the participants by hinting to increase the settings and/or taking new pictures. This problem also occurred in the previous tests and could not be resolved and should be focused on by future researchers.

The middle and large object were annotated by all participants without error on the first try.

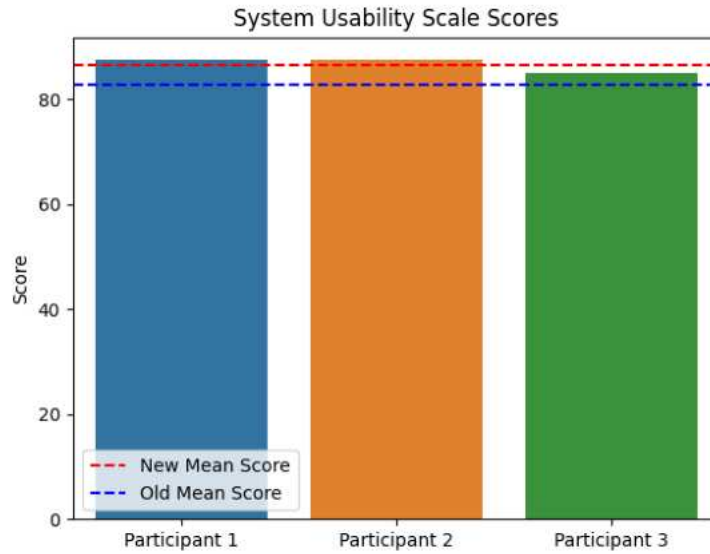


Figure 5.2.: SUS scores with comparison to old SUS score

5.2.3. System usability scale

This usability scale consists of ten statements and five agreement levels as answers. The survey is designed to evaluate the usability of the app. The average SUS score was approximately 86.67 (see 5.2) which is above the average SUS score of 68 indicating an excellent usability score [Sau11]. This score indicates that the newly added features and UI elements did not influence the usability as the mean SUS value before the changes was 82.83.

5.2.4. NASA task load index

As part of the testing process, the participants were requested to complete the NASA Task Load Index (TLX), which assesses the level of workload associated with a given task. Specifically, in our study, the task under evaluation was the application being tested. The TLX comprises six statements and twenty-one response levels, with 0 indicating the most desirable response level, and 20 representing the least desirable response level. Like in Klusmanns study the weightings of the questions were determined by the test supervisor to get comparable results. The higher the weight the stronger the impact to the final score. Temporal Demand (1), Performance (2), Effort (2), Mental Demand (3), Physical Demand (3) and Frustration (4). The mean TLX was 24.67

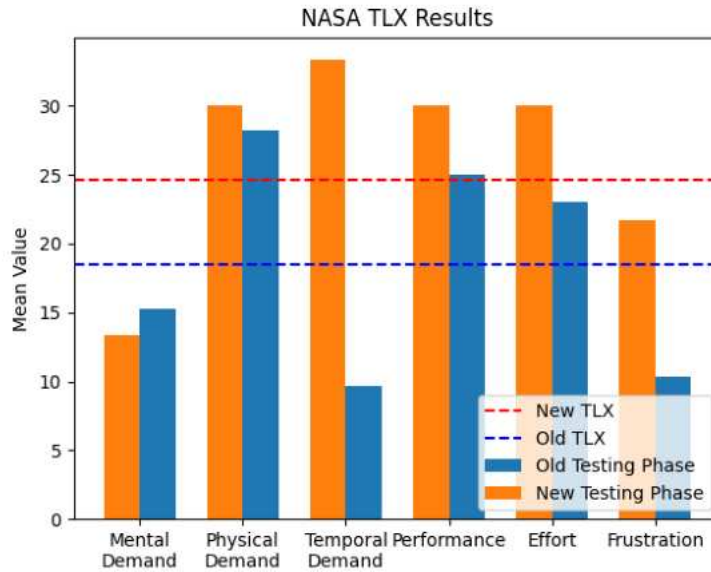


Figure 5.3.: NASA TLX with comparison to old TLX

which is higher than in the previous testing (18.52). The Mental (13.33), Physical (30) and Performance (30) task load mean values were similar to the previous results (15.3, 28.27, 25). Bigger differences could be determined within the Temporal (33.33), Effort (30) and Frustration (21.67) aspect of the TLX. Before these values were (9.67, 23.0, 10.33) (see 5.3). These differences could be a result from the difficulties every participant had with the small object. The biggest difference was the temporal demand. This may have resulted from the fact that the participants knew that their completion time was noted and then they felt more of a hurry to finish the task. One participant also stated that he doesn't want to be the slowest so he hurried. Future test could explore the difference it makes whether you tell the participant that the time is noted or not. Also one could test how rushing this task would affect the resulting Vol. E.g. giving the participant 60 seconds for each task and having the supervisor rate the quality of the reconstruction.

5.2.5. Application survey

This survey by [Klu23] contained 14 statements to analyze how design decisions, UI elements and the application structure influenced the user experience. The participants could choose from five agreement levels where 5 is the most preferable answer.

The statements are:

1. I understood how to use the application after the demonstration
2. Using the iPad or Apple Pencil did not cause any discomfort while performing my tasks
3. It was easy to navigate and use the application's camera tab
4. It was easy to navigate and use the application's drawing tab
5. It was easy to mark the object of interest in previously taken images
6. My markings were clearly visible to me in the pictures I had taken
7. It was easy to navigate and use the application's gallery tab
8. It was easy to delete and edit previous image markings
9. It was easy to initiate the 3D reconstruction process
10. The resulting volume of interest was clearly visible to me in the real-world environment
11. It was easy to navigate and use the settings tab (including adjusting the settings accordingly)
12. The button's images matched their respective functions
13. The application's performance did not interfere with the performance of my tasks
14. The application's design is consistent and logical in regard to the work flow of my tasks

(Note: For statement two the participants were advised that in this case instead of iPad it is a tablet device and instead of an apple pencil it was a touch-pen)

The final question asks the participants to rate their overall experience while using the application from 1 to 10 which resulted in a new mean score of 9 (see 5.5).

The similar overall mean values (see 5.4) indicate that the newly added UI elements and features did not affect the user experience in a negative way. Nevertheless, this must be researched and possibly confirmed in a larger scaled study.

5. Evaluation

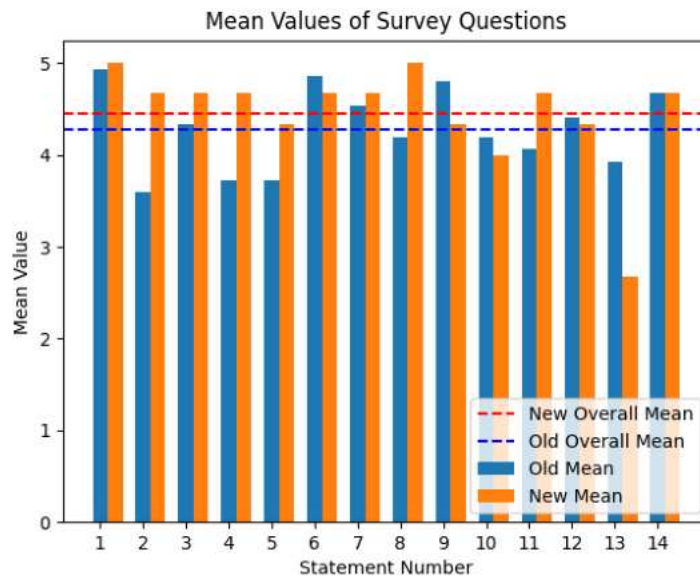


Figure 5.4.: Statement scores with comparison to previous survey

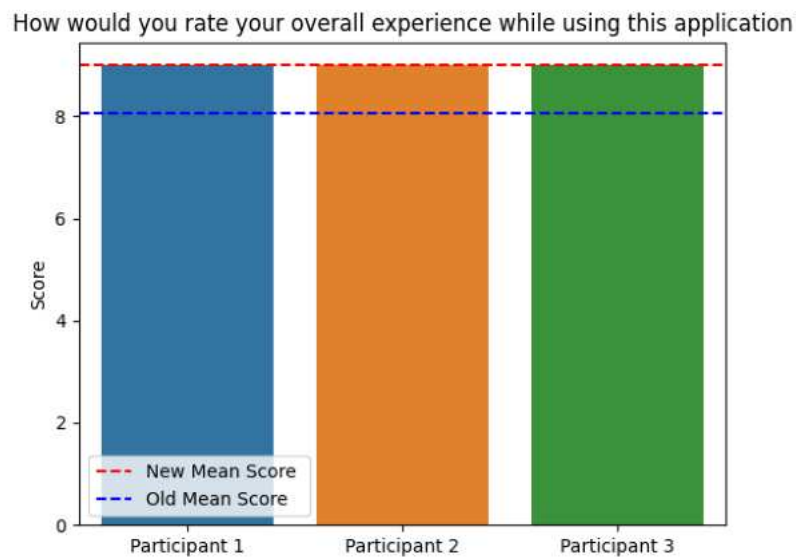


Figure 5.5.: Application rating with comparison to previous rating

5.2.6. Interview

The interview was held after each participant finished to fill out all surveys. In the following the answers to each question get summed up.

1. *What was your first impression of the application?*

The app has a simple design which is easy to understand even as layman.

2. *Is there anything that you particularly liked about this application?*

There is little room for errors, because there are not unnecessarily many buttons and you need only a few steps to create the VoI. Furthermore, the visualization of the VoI was very pleasing.

3. *Is there anything that you particularly disliked about this application? Did you have any difficulties using it?*

All three participants stated that they had problems with the small, occluded object. Another negative aspect that got mentioned was the bad handling of the tablet because of its cover.

4. *Are there any features or user interface elements missing that you think should be added?*

- **Gallery UI:** If the user goes to the gallery tab the first image should directly displayed instead of having to click on it. Furthermore the selected image should be highlighted in the image list.
- **Intersect Button:** The icon on the intersect button is not intuitive
- **Object Detection:** The system should identify and annotate objects by itself and the user can then select the Object/Volume of Interest needed
- **Automated Initial Voxelcount:** Based on the complexity of the annotations the system should automatically increase or decrease the setting. Furthermore if the Smart Cube Adjustment is successful but the visualization fails, the application should increase the setting automatically and try again until a certain threshold is reached.

5. *Are there any features or user interface elements present that you think should be removed?*

All participants answered with No.

6. *Do you have any other comments about your experience that you would like to share?*

One user stated that using the app was a fun experience. The others had no more comments.

5.2.7. Summary

In summary the small scale user study showed similar results on most surveys, indicating that the new features and UI elements do not interfere with the systems usability. But as previously stated, this was not a representative study, so these indications have to be verified by future researchers. The higher NASA TLX is a hint that if the issue with small, occluded objects is not addressed, this score could rise even higher, meaning this is a very important issue to focus on for future research.

It should be mentioned that in preparation for this study it was planned to use a different tablet device (Samsung Galaxy Tab S2: SM-T813). The device was replaced by a newer one, while preparing the device for the user study it became apparent that annotation was only possible with the old device when the marker was in the field of view of the camera. For extended tracking meaning the marker is not visible after the initial scan, the camera and the internal sensors are too inadequate. Therefore, it is recommended that future researchers use a comparable or even better device as above (see 5.1) or in Klusmann's usability tests (iPad Pro 12.9" (2021)).

Also while testing the app, it was noticed that on devices with a higher aspect ratio, e.g. 20:9 (Google Pixel 7), the 3D volume becomes either narrower or wider depending on whether you use the app in portrait or landscape mode. On devices with a smaller aspect ratio, e.g. 16:10 (5.1) the distortion was not as severe, but still present.

6. Future Work

The research presented in this thesis has generated new knowledge about communication interfaces for this augmented reality application, and future work could build upon this foundation to explore additional questions or expand the scope of the study.

Future researchers could improve the annotation process for small, occluded and/or difficult to access objects to expand the current limitations of the application. In addition, they could investigate the previously mentioned distortion problem with high aspect ratios to make the app usable on more devices.

Another issue that should be addressed is the handling of the device. Future researchers could explore combinations of different devices and different arrangements of the buttons to make the application easier to use.

Another topic that future researchers could address is the scalability of the system. At the moment the application only supports a single marker environment with one VoI at a time. In order to meet the industrial requirements the application should be able to identify multiple markers and retrieve their ID. So that when generating the VoI the respective marker ID gets sent to the server with the glTF file. The users should also be able to scan a marker and then retrieve all VoIs from the server associated with this marker (see figure 2.2, 2.3).

Future research could also include the exploration of other mesh generation algorithms for example the Marching Tetrahedra algorithm.

To explore better communication interfaces between the application and the server, one could research the usage of glb files instead of the combination of a glTF and a bin file for the VoI transfer.

7. Conclusions

In conclusion, this thesis project aimed to achieve three goals: firstly, to implement an efficient interface for transferring and processing VoI and its pose using the .glTF data format; secondly, to incorporate the Marching cubes algorithm into the application, enabling a cleaner and less memory-intensive representation of the VoI; and finally, to investigate the impact of incorporating these features and switching from an iPad to an Android device on test scores and other metrics.

Through the development and implementation of appropriate software tools and testing procedures, as well as the analysis and interpretation of the resulting data, these goals were successfully accomplished. Specifically, the visualization of the VoI was improved by adding the Marching Cubes algorithm Plugin [Haw22] and making it usable with an adapter. While implementing this adapter, the intersection algorithm got changed from using a Vector3 List to a 3D boolean array to represent the valid voxels. Additionally the algorithm was changed to check the voxel centers instead of all 8 corners of the voxel, resulting in a faster intersection process, smaller file sizes, and better visualization of the VoI.

Additionally, a communication interface was established and implemented (see figures 2.2, 2.3), allowing for the Volume of Interest created by the user to be exported and imported [Att22], [Lon23] using the widely supported glTF file format. To simulate and test the interface, a local host server was implemented with Python, demonstrating that the proposed communication interface is effective and can be integrated into an industrial setting where the VoI is processed further.

Furthermore, all tests conducted by Klusmann [Klu23] were redone, and the results showed promising results indicating nearly no negative influence of the newly added features and UI elements on the user experience. These findings have important implications for the field of 3D reconstruction, providing valuable insights into the potential benefits of utilizing the .glTF data format and Marching cubes algorithm in VoI applications, as well as the impact of device type on performance metrics.

Overall, this project has contributed to the advancement of knowledge in this area and has laid the groundwork for future research and development in this field.

A. Added user interface elements

- An upload and a download button (see purple arrow buttons in screenshots) were added to send and receive the created VoI files to the test server.
- A button to remove the created or received VoIs from the scene was added. Before you had to close and restart the app to remove them.
- For the sake of restarting the app a button was added, so that the user does not have to close and reopen the app which results in a better workflow.
- An UI element to indicate whether a marker is tracked (green icon) or not (red icon) was added (see top right corner of app screenshots). In addition to this the condition was added that photos can only be taken if a marker is tracked by the application, which resulted in fewer failures in the reconstruction process. Because now every image definitely has the pose information needed for the calculation. Before adding this condition this was a very regular source of errors.
- For better troubleshooting more error messages were added to the application
 - *No marker tracked!:* In addition to ui element indicating the tracking status, there is also an error message if the user tries to take a photo without a marker being tracked.
 - *Smart Cube Adjustment failed!:* If the angles from the photos were too similar to each other, no 3D bounding box can be calculated thus no intersection can happen.
 - *No valid voxels found!:* If the SCA succeeded there it still can happen that no voxels were found for example because the voxelsize is bigger than the actual object. The user then has to increase the start voxel count and/or the iteration count.
 - *Generate VoI before sending it!:* This message appears if the user tries to upload nothing to the server.
 - Additionally several error messages for the server communication were added.

A. Added user interface elements

- The application now supports portrait mode, as before only landscape mode was supported which makes it easier to hold the device in one hand and navigate with the touch pen in the other.

List of Figures

2.1. Spatial relationship graph for 3D annotation	7
2.2. Communication protocol for glTF sending.	7
2.3. Communication protocol for glTF downloading.	7
4.1. In-app screenshots with old, new and merged VoIs	13
4.2. VoI with cube prefabs	14
4.3. VoI with Marching Cubes algorithm	15
4.4. Scene with server answer	16
5.1. Test machine with the marker (green) and three different sized objects of interest: small (red), medium (yellow), large (purple)	21
5.2. SUS scores with comparison to old SUS score	23
5.3. NASA TLX with comparison to old TLX	24
5.4. Statement scores with comparison to previous survey	26
5.5. Application rating with comparison to previous rating	26

List of Tables

5.1. Test device information 19

Bibliography

- [Att22] A. Atteneder. *glTFast*. <https://github.com/atteneder/glTFast>. Accessed January 04, 2023. 2022.
- [Azu97] R. T. Azuma. "A Survey of Augmented Reality." In: *Presence: Teleoperators and Virtual Environments* 6.4 (Aug. 1997), pp. 355–385. DOI: 10.1162/pres.1997.6.4.355. eprint: <https://direct.mit.edu/pvar/article-pdf/6/4/355/1623026/pres.1997.6.4.355.pdf>.
- [BD12] M. Billinghurst and A. Duenser. "Augmented Reality in the Classroom." In: *Computer* 45.7 (2012), pp. 56–63. DOI: 10.1109/MC.2012.111.
- [Bro+96] J. Brooke et al. "SUS-A quick and dirty usability scale." In: *Usability evaluation in industry* 189.194 (1996), pp. 4–7.
- [Che+00] G. K. Cheung, T. Kanade, J.-Y. Bouguet, and M. Holler. "A real time system for robust 3D voxel reconstruction of human motions." In: *Proceedings IEEE Conference on Computer Vision and Pattern Recognition. CVPR 2000 (Cat. No. PR00662)*. Vol. 2. IEEE. 2000, pp. 714–720.
- [Cho+18] Y. Chong, D. K. Sethi, C. H. Y. Loh, and F. Lateef. "Going Forward with Pokemon Go." In: *J Emerg Trauma Shock* 11.4 (2018), pp. 243–246. DOI: 10.4103/JETS.JETS_87_17.
- [Ech+08] F. Echtler, M. J. Huber, D. Pustka, P. Keitler, and G. Klinker. "Splitting the Scene Graph - Using Spatial Relationship Graphs Instead of Scene Graphs in Augmented Reality." In: *GRAPP 2008, Proceedings of the Third International Conference on Computer Graphics Theory and Applications, Funchal, Madeira, Portugal, January 22-25, 2008*. Ed. by J. Braz, N. J. Nunes, and J. M. Pereira. INSTICC - Institute for Systems, Technologies of Information, Control, and Communication, 2008, pp. 456–459.
- [Har86] S. G. Hart. "NASA task load index (TLX)." In: (1986).
- [Haw22] J. Hawkins. *Marching-Cubes*. <https://github.com/Scrawk/Marching-Cubes>. Accessed January 16, 2023. 2022.

- [KLM20] S. Kwon, H. Lee, and D. Mun. "Semantics-aware adaptive simplification for lightweighting diverse 3D CAD models in industrial plants." In: *Journal of Mechanical Science and Technology* 34 (2020), pp. 1289–1300. doi: 10.1007/s12206-020-0228-y.
- [Klu23] J. S. Klusmann. "Volume of Interest Annotation Methods in Industrial Use Cases using Augmented Reality." Technical University of Munich, 2023.
- [LC87] W. E. Lorensen and H. E. Cline. "Marching Cubes: A High Resolution 3D Surface Construction Algorithm." In: *SIGGRAPH Comput. Graph.* 21.4 (Aug. 1987), pp. 163–169. issn: 0097-8930. doi: 10.1145/37402.37422.
- [Lin] J. Linietsky. *Why we should all support glTF 2.0 as THE standard asset exchange format for game engines*. <https://godotengine.org/article/we-should-all-use-gltf-20-export-3d-assets-game-engines/>. Accessed: March 8, 2023.
- [Lon23] R. Long. *UnityGLTF*. <https://github.com/prefrontalcortex/UnityGLTF>. Accessed January 04, 2023. 2023.
- [Mar22] Marco Hutter. *glTF 2.0 API Reference Guide*. Tech. rep. Accessed: February 28, 2023. Khronos Group, 2016-2022.
- [MSZ17] R. Miao, J. Song, and Y. Zhu. "3D geographic scenes visualization based on WebGL." In: *2017 6th International Conference on Agro-Geoinformatics*. 2017, pp. 1–6. doi: 10.1109/Agro-Geoinformatics.2017.8046999.
- [NL93] J. Nielsen and T. K. Landauer. "A Mathematical Model of the Finding of Usability Problems." In: *Proceedings of the INTERACT '93 and CHI '93 Conference on Human Factors in Computing Systems*. CHI '93. Amsterdam, The Netherlands: Association for Computing Machinery, 1993, pp. 206–213. isbn: 0897915755. doi: 10.1145/169059.169166.
- [PTC21] PTC Inc. *Device Tracking*. <https://library.vuforia.com/environments/device-tracking>. Accessed: February 28, 2023. 2011-2021.
- [RK21] L. Rudolph and G. Klinker. "As-built Industrial Scene Reconstruction based on Photogrammetry and Prior-Knowledge for Extended Reality Scenarios." In: *2021 IEEE International Symposium on Mixed and Augmented Reality Adjunct (ISMAR-Adjunct)*. 2021, pp. 480–482. doi: 10.1109/ISMAR-Adjunct54149.2021.00113.

- [Rud+22] L. Rudolph, D. Pantförder, F. Palmas, M. Fischer, P. Niermann, G. Klinker, and B. Vogel-Heuser. "Maintenance in Process Industries with Digital Twins and Mixed Reality: Potentials, Scenarios and Requirements." In: *2022 IEEE International Conference on Industrial Engineering and Engineering Management (IEEM)*. 2022, pp. 0474–0481. doi: 10.1109/IEEM55944.2022.9989826.
- [Sau11] J. Sauro. *Measuring Usability with the System Usability Scale (SUS)*. <https://measuringu.com/sus/>. accessed March 27, 2023. 2011.
- [SBN16] A. Schilling, J. Bolling, and C. Nagel. "Using GITF for Streaming CityGML 3D City Models." In: *Proceedings of the 21st International Conference on Web3D Technology*. Web3D '16. Anaheim, California: Association for Computing Machinery, 2016, pp. 109–116. ISBN: 9781450344289. doi: 10.1145/2945292.2945312.
- [SK08] B. Schwerdtfeger and G. Klinker. "Supporting order picking with Augmented Reality." In: *2008 7th IEEE/ACM International Symposium on Mixed and Augmented Reality*. 2008, pp. 91–94. doi: 10.1109/ISMAR.2008.4637331.
- [Sti22] F. Stieglbauer. "Silhouette-Based Segmentation of 3D Objects for Industrial Use Case." In: (2022).
- [Uni] D. University. *C# Primitive Datatypes*. <https://condor.depaul.edu/sjost/nwdp/notes/cs1/CSDatatypes.htm>. Accessed March 21, 2023.
- [Zhu+14] E. Zhu, A. Hadadgar, I. Masiello, and N. Zary. "Augmented reality in healthcare education: an integrative review." In: *PeerJ* 2:e469 (2014). doi: 10.7717/peerj.469.